1 Projeto Sistema de Arquivos:

1.1 Introdução

Um aluno de Eng. de Computação recebeu a tarefa de criar um sistema de arquivos inspirado no inode no Unix para dispositivos embarcados simples. O sistema de arquivos questão deve armazenar seus dados num chip de memória serial i2c **AT24C512**, **AT24C256** [1].

O sistema de arquivos tem um layout apresentado em aula (Fig.1) e cada entrada de arquivo é representada como um inodo (Fig.2).

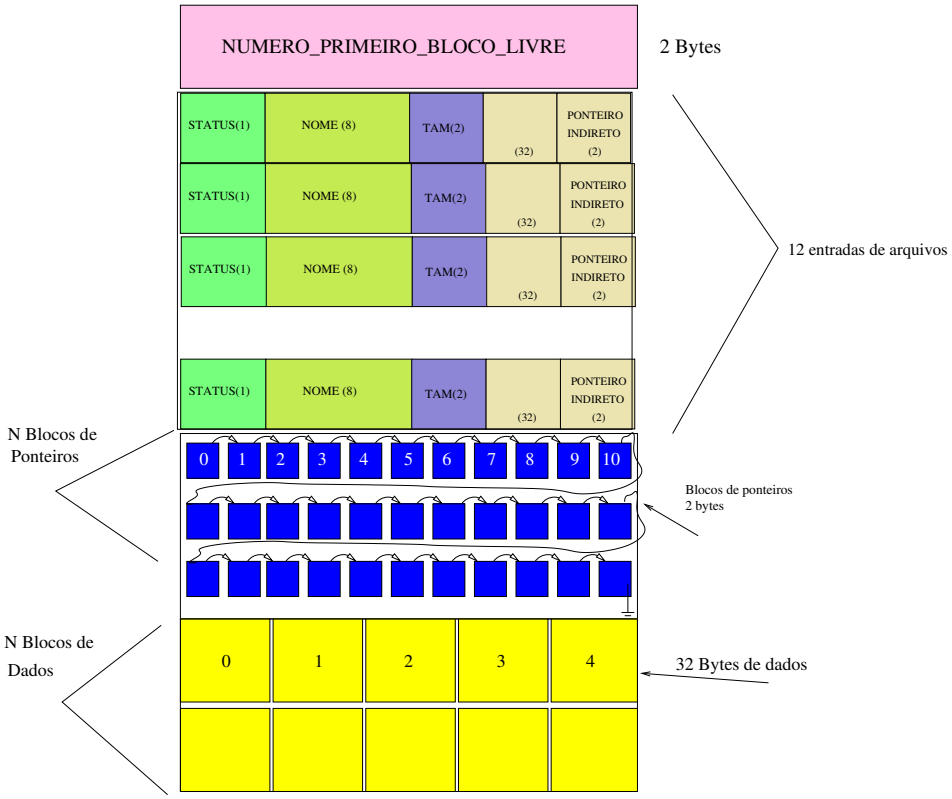


Figure 1: Layout

1.1.1 Como funciona:

Existe um cabeçalho de 2 Bytes que indica qual é o número do primeiro bloco de dados livre. Após, existem 12 entradas de arquivos (inodos). Cada entrada de arquivo é composta por campos (Status - Indica se a entrada está livre ou ocupado, Nome - 8 letras para o nome do arquivo, tamanho - quantos bytes o arquivo ocupa, bloco direto (32 bytes de dados), bloco indireto (é o endereco de um bloco cujo conteúdo são enderecos para blocos de dados). Em azul temos os blocos de ponteiros e abaixo destes os blocos de dados. Os blocos de ponteiros possuem 2 Bytes cada e iniciamente foram organizados para criar uma lista encadeada. Um bloco armazena nos seus 2 Bytes o endereço o próximo bloco livre. Existe uma relação entre cada bloco de ponteiro e o bloco de dados. O bloco de ponteiro 0 está relacionado ao bloco de dados 0.

1.1.2 Criação de uma entrada de arquivo:

Deve-se procurar por uma entrada livre (entrada com o campo status valendo 0) Quando for encontrada, marca-se a entrada como ocupada (status valendo 1).

1.1.3 Alocação:

Toda a vez que um bloco precisa ser alocado, verifica-se o cabeçalho e descobre-se qual é o primeiro bloco livre que seja utilizado. Depois, acessando o número desse bloco na lista encadeada, descobre-se qual o número do próximo bloco e esse número será o novo primeiro bloco livre no cabeçalho.

[1] https://pdf1.alldatasheet.com/datasheet-pdf/view/56068/ATMEL/AT24C512.html

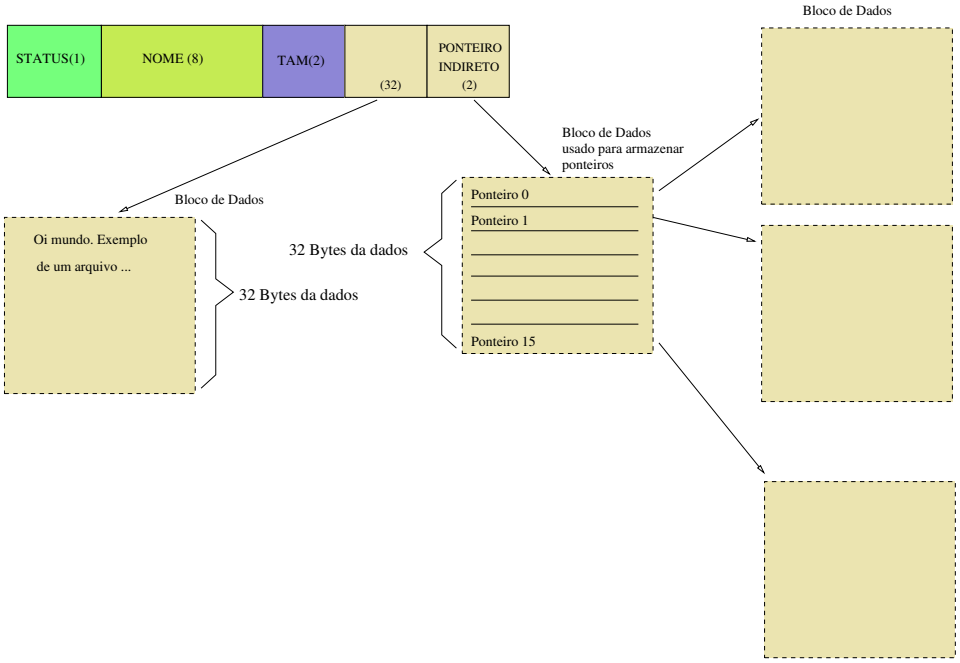


Figure 2: inodo

1.1.4 Remoção de um arquivo:

Para remover um arquivo é preciso desalocar todos os blocos de dados pertencentes a um arquivo e depois disso, tornar a entrada de arquivo disponível. Os blocos desalocados devem ser inseridos na lista encadeada.

O sistema de arquivos pode ser imaginado como um sistema em camadas, tal como mostrado na figura 3.

1.2 O que é solicitado ?

Implemente o código do sistema de arquivos para torná-lo funcional e mais especificamente as funções mostradas na tabela abaixo. Teste usando o exemplo em C mostrado:

PS: Você pode implementar melhorias no sistema de arquivo, como por exemplo um mecanismo de cache para evitar perda de tempo acessando posições de memória que já se encontram num buffer.

Função	comentários
fopen	implemente apenas os modos de abertura r+ e w
fread / fwrite	implemente essas funções para ler e escrever vários bytes de uma só vez. Note que você não pode fazer chamadas sucessivas a fgetc/fputc para implementar estas funções em virtude do overhead envolvido
feof	implemente esta função
ftell	implemente esta função
fclose	implemente esta função

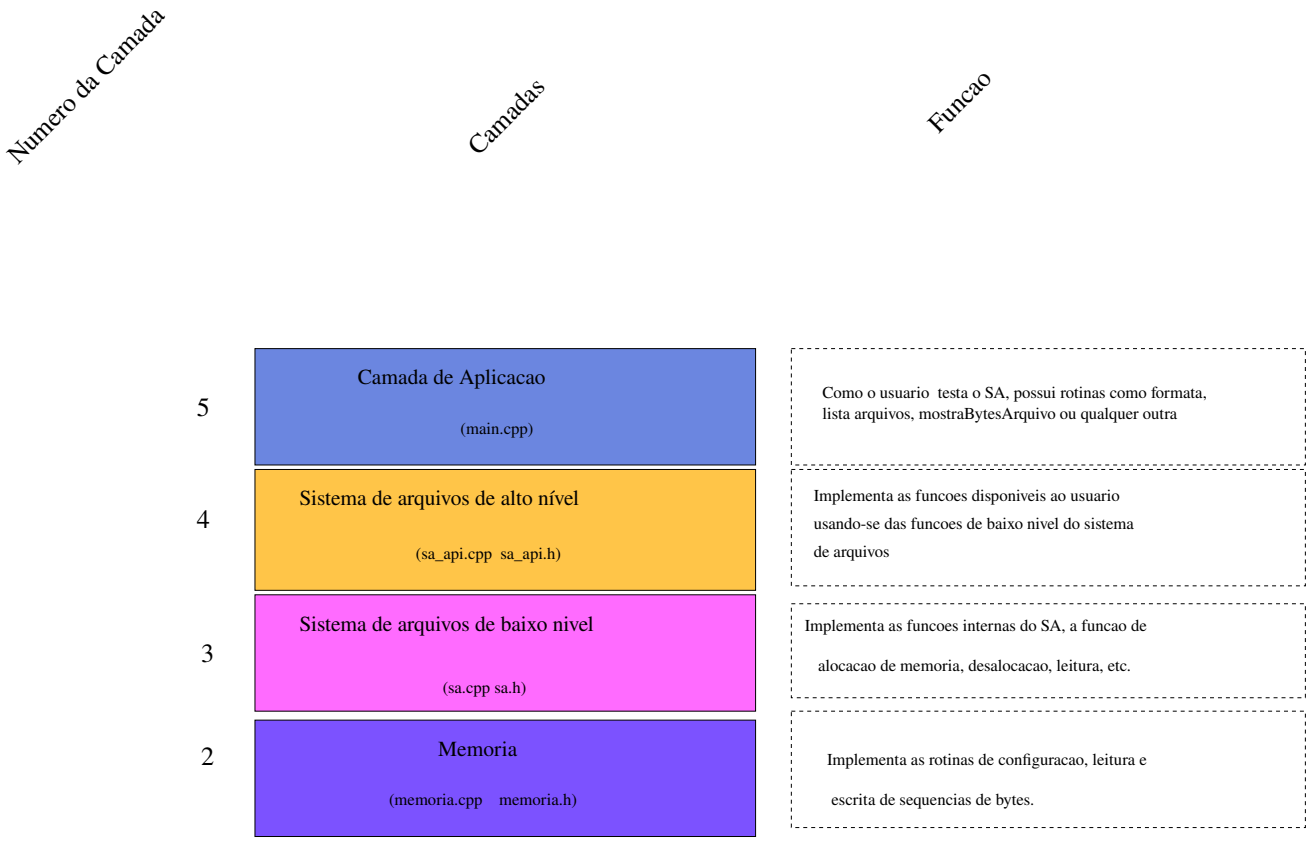


Figure 3: Sistema visto em camadas

```
1
2  #include <stdio.h>
3  #include <string.h>
4  #include "sa_api.h"
5
6  int main (void)
7  {
8    char buffer[70],a;
9    ARQUIVO *A, *B, *C;
10
11   printf("Tecle [ENTER] para iniciar\r");
12   while (1)
13   {
14     a=getchar();
15     if (a==13) break;
16   }
17   printf("OK \r");
18
```

```
  inicializa_SA();
  formata_SA(); // realizado apenas 1 vez

  A = meu_fopen ("UFSC","w");
  strcpy(buffer, "Oi mundo, teste de mensagemXXX");
  meu_fwrite(A,(uint8_t *)buffer, strlen(buffer));
  meu_fclose(A);

  B = meu_fopen ("Fabio","w");
  strcpy(buffer, "Oi mundo, teste de mensagemXXX");
  meu_fwrite(B,(uint8_t *)buffer, strlen(buffer));

  strcpy(buffer, " Prof: FABIO");
  meu_fwrite(B,(uint8_t *)buffer, strlen(buffer));

  strcpy(buffer, " de la Rocha UFSC Ararangua");
  meu_fwrite(B,(uint8_t *)buffer, strlen(buffer));
  meu_fclose(B);

  C=meu_fopen("Fabio","r");
  meu_fseek(C,19,SEEK_SET);
  meu_fread(C,(uint8_t *)buffer,8);
  buffer[8]=0;
  printf("Lido=%s\r",buffer);



  lista_todos();

  while(1)
  {
  }
  return 0;

}
```

2 Implementação durante o ciclo de desenvolvimento

A implementação final deve ser realizada utilizando o ESP32, o framework da idf e o chip de memória EEPROM i2c. Porém, o ciclo de desenvolvimento (compilação, flash e testes pela serial) resulta em tempo extra durante o desenvolvimento e mais problemas de *debug*.

2.1 Implementação do código no Linux

sugere-se implementar o SA e a aplicação de testes no Linux e emular as chamadas de acesso à memória.

Ou seja, digamos que o código do ESP32 tenha chamadas `memoria_init`,`memoria_fwrite` e `memoria_fread`, você pode criar essas funções no linux com o mesmo nome mas que ao invés de acessar bytes da memória, acessa bytes de um arquivo no linux (onde o tal arquivo representa o conteúdo da memória)

```

void memoria_init()
{
  printf("Chamou a versao de mem.init do linux\n");
  printf("Cria um arquivo para emular a memoria\n");


```

```
8   f = fopen ("PARTICAO.BIN","r+");
9   if (f==NULL)
10  {
11    f = fopen ("PARTICAO.BIN","w+");
12    if (f==NULL)
13    {
14      printf("Impossivel criar aquivo de particao\n");
15    }
16  }
17
18 }
19
20 void memoria_fwrite(void *buffer, uint16_t posicao, uint16_t tam)
21 {
22   printf("Chamou a versao de mem.write do linux\n");
23   fseek(f, posicao, SEEK_SET);
24   fwrite(buffer, tam, 1, f);
25 }
26
27 void memoria_fread(void *buffer, uint16_t posicao, uint16_t tam)
28 {
29     printf("Chamou a versao de mem.read do linux\n");
30     fseek(f, posicao, SEEK_SET);
31   fread(buffer, tam, 1, f);
32 }
```

Como a memória está sendo emulada utilizando um arquivo, pode-se usar ferramentas como `hexdump` para examinar o conteúdo do tal arquivo e assim descobrir se os dados foram salvos corretamente. Você pode inclusive criar sua propria ferramenta para facilitar a visualização do conteúdo do arquivo.